Anjuman-I-Islam's

AKBAR PEERBHOY COLLEGE OF COMMERCE & ECONOMICS

M. S. Ali Road, Do Taaki, Grant Road (E), Mumbai-8.

PRACTICAL JOURNAL

SUBMITTED TO

T.Y. B.Sc. (Data Science)

Semester - VI

Machine Learning

University of Mumbai

By

Aamir Asif Shaikh

Under the guidance of

Prof. Shahid Ansari

DEPARTMENT OF DATA SCIENCE

AKBAR PEERBHOY COLLEGE OF COMMERCE & ECONOMICS

Academic Year (2025 – 2026)

Anjuman-I-Islam's

"AKBAR PEERBHOY COLLEGE OF COMMERCE & ECONOMICS"

M.S. Ali Road, Mumbai - 400008

CERTIFICATE

This is a certify that **Aamir Asif Shaikh** of The class **T.Y.B. Sc - Data Science** Roll No: **46** University Exam Seat No: _____has satisfactorily completed the Practical Journal in the subject of **Machine Learning** as Prescribed in the syllabus of **Third Year Bachelor of Science in Data Science** During the academic year 2025 – 2026

B.Sc. Data Science Coordinator (Signature)	**External Examiner (Signature)**	**Lecturer-in-Charge (Signature)**

Practical Index

1.	a) Implementing simple linear regression b) Implementing linear regression (e.g. to predict housing prices based on features such as size and location).		
2.	Using logistic regression (e.g. to predict whether a credit card transaction is fraudulent or not).		
3.	Implementing a K-Nearest Neighbor (KNN) algorithm (e.g. to classify handwritten digits).		
4.	Building a decision tree model using the ID3 algorithm (e.g. to predict whether a customer will churn or not).		
5.	Building a Naïve Bayes classifier (e.g. to classify movie reviews as positive or negative sentiment).		
6.	Developing a Support Vector Machine (SVM) model (e.g. to classify email messages as spam or not spam).		
7.	Implementing the K-means clustering algorithm (e.g. to identify distinct clusters in a customer demographic dataset).		
8.	Applying hierarchical clustering (e.g. to group customer segments based on their purchasing behavior).		
9.	Utilizing Principal Component Analysis (PCA) for dimensionality reduction to improve the efficiency and interpretability of a model.		
10.	Evaluating a classification model using metrics such as accuracy, precision, recall, and F1 score.		

Practical No.1-A: Implementation of Simple Linear Regression

Step-1: Import required libraries

```
import random as r
from sklearn.linear_model import LinearRegression

print("Aamir Asif Shaikh - 46")
print("====================================")
print("All the required libraries are install")
```

```
Aamir Asif Shaikh - 46
====================================
All the required libraries are install
```

Step-2: Read Dataset

```
print("Aamir Asif Shaikh - 46")
print("====================================")
features_set = []
target_set = []

count = 0
for i in range(0,200):

  x = r.randint(1,10)
  features_set.append([x])

  y = 2*x + 1
  target_set.append([y])
  count = count + 1

print("Total Records :" ,count)
print("Features Set")
print(features_set)
print("Target Set")
print(target_set)
```

```
Aamir Asif Shaikh- 46
====================================
Total Records : 200
Features Set
[[5], [7], [4], [5], [6], [9], [10], [7], [1], [4], [1], [7], [7],
[4], [8], [6], [7], [4], [5], [10], [8], [8], [1], [3], [7], [8], [1],
[9], [7], [2], [5], [6], [8], [9], [3], [5], [3], [4], [8], [7], [10],
[1], [5], [8], [6], [2], [4], [1], [10], [10], [2], [8], [3], [9],
[10], [5], [5], [10], [4], [6], [7], [7], [9], [6], [5], [4], [7],
[9], [6], [10], [8], [3], [1], [10], [5], [4], [1], [5], [5], [4],
```

```
[8], [3], [7], [2], [3], [4], [9], [7], [1], [3], [6], [9], [5], [4],
[7], [9], [8], [10], [1], [10], [1], [3], [1], [10], [6], [10], [4],
[6], [7], [10], [5], [8], [7], [10], [9], [6], [8], [7], [1], [7],
[7], [6], [2], [9], [7], [6], [9], [3], [1], [1], [4], [10], [8], [4],
[5], [6], [9], [4], [2], [6], [6], [7], [7], [5], [6], [5], [9], [3],
[10], [8], [9], [9], [6], [6], [5], [6], [9], [2], [9], [6], [2], [1],
[1], [10], [2], [3], [6], [6], [6], [9], [5], [3], [5], [4], [10],
[8], [7], [5], [4], [7], [7], [7], [10], [9], [3], [9], [2], [9], [7],
[9], [1], [2], [8], [1], [8], [5], [6], [1], [7], [7]]
Target Set
[[11], [15], [9], [11], [13], [19], [21], [15], [3], [9], [3], [15],
[15], [9], [17], [13], [15], [9], [11], [21], [17], [17], [3], [7],
[15], [17], [3], [19], [15], [5], [11], [13], [17], [19], [7], [11],
[7], [9], [17], [15], [21], [3], [11], [17], [13], [5], [9], [3],
[21], [21], [5], [17], [7], [19], [21], [11], [11], [21], [9], [13],
[15], [15], [19], [13], [11], [9], [15], [19], [13], [21], [17], [7],
[3], [21], [11], [9], [3], [11], [11], [9], [17], [7], [15], [5], [7],
[9], [19], [15], [3], [7], [13], [19], [11], [9], [15], [19], [17],
[21], [3], [21], [3], [7], [3], [21], [13], [21], [9], [13], [15],
[21], [11], [17], [15], [21], [19], [13], [17], [15], [3], [15], [15],
[13], [5], [19], [15], [13], [19], [7], [3], [3], [9], [21], [17],
[9], [11], [13], [19], [9], [5], [13], [13], [15], [15], [11], [13],
[11], [19], [7], [21], [17], [19], [19], [13], [13], [11], [13], [19],
[5], [19], [13], [5], [3], [3], [21], [5], [7], [13], [13], [13],
[19], [11], [7], [11], [9], [21], [17], [15], [11], [9], [15], [15],
[15], [21], [19], [7], [19], [5], [19], [15], [19], [3], [5], [17],
[3], [17], [11], [13], [3], [15], [15]]
```

Step 3 :Model Creation

```
print("Aamir Asif Shaikh - 46")
print("====================================")
model = LinearRegression()
model.fit(features_set,target_set)

print(model)
print("Model is trained")
```

```
Aamir Asif Shaikh - 46
====================================
LinearRegression()
Model is trained
```

Step 4: Test Model

```
print("Aamir Asif Shaikh - 46")
print("====================================")
test_data =[[0]]
```

```
prediction = model.predict(test_data)
print("Your prediction",prediction)
```

```
Aamir Asif Shaikh - 46
=====================================
Your prediction [[1.]]
```

Practical No.1-B: Implementing linear Regression (e.g: To predict ousing prices based on Features)

Step-1 Import required LibrMaries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

print("Aamir Asif Shaikh - 46")
print("====================================")
print("All the Required libraries are imported!!")
```

```
Aamir Asif Shaikh- 46
====================================
All the Required libraries are imported!!
```

Step-2: Read Dataset

```
data = {
    "Size_sqft":[850,900,1200,1000,1500,1800,2000,2200,2500,3000],
    "Rooms": [2,2,3,3,3,4,4,4,5,5],
    "Location_Score": [3,4,4,5,5,4,5,5,4,5],
    "Price" :
[200000,220000,240000,280000,320000,360000,400000,430000,480000,550000
],
}

df = pd.DataFrame(data)
print("Dataset :")
print(df)
```

```
Dataset :
   Size_sqft  Rooms  Location_Score    Price
0        850      2               3  200000
1        900      2               4  220000
2       1200      3               4  240000
3       1000      3               5  280000
4       1500      3               5  320000
5       1800      4               4  360000
6       2000      4               5  400000
7       2200      4               5  430000
8       2500      5               4  480000
9       3000      5               5  550000
```

Step-3: Divide data into feature and target

```
print("Aamir Asif Shaikh - 46")
print("====================================")

x = df[["Size_sqft","Rooms","Location_Score"]]
y = df[["Price"]]

print("Feature :")
print(x)
print("Target")
print(y)
```

```
 Aamir Asif Shaikh- 46
====================================
Feature :
   Size_sqft  Rooms  Location_Score
0        850      2               3
1        900      2               4
2       1200      3               4
3       1000      3               5
4       1500      3               5
5       1800      4               4
6       2000      4               5
7       2200      4               5
8       2500      5               4
9       3000      5               5
Target
    Price
0  200000
1  220000
2  240000
3  280000
4  320000
5  360000
6  400000
7  430000
8  480000
9  550000
```

Step-4 Split Dataset into training and Testing

```
x_train,x_test,y_train,y_test =
train_test_split(x,y,test_size=0.2,random_state=42)
print("X_Train Data")
print(x_train)
print("X_Test Data")
print(x_test)
```

```
X_Train Data
    Size_sqft  Rooms  Location_Score
5        1800      4               4
```

```
0         850      2                 3
7        2200      4                 5
2        1200      3                 4
9        3000      5                 5
4        1500      3                 5
3        1000      3                 5
6        2000      4                 5
X_Test Data
    Size_sqft  Rooms  Location_Score
8        2500      5               4
1         900      2               4
```

Step-5 : Model Creation and Training

```
model = LinearRegression();
model.fit(x_train,y_train)
print("Aamir Asif Shaikh - 46")
print("====================================")
print("Model Name :",model)
print("Model is trained!!")
```

```
Aamir Asif Shaikh- 46
====================================
Model Name : LinearRegression()
Model is trained!!
```

Step-6 :Test Model

```
prediction = model.predict(x_test)
print("Prediction Output")
print(prediction)
```

```
Prediction Output
[[458163.53485022]
 [216644.65389925]]
```

Practical 2: Using logistic regression (e.g. to predict whether a credit card transaction is fraudulent or not).

Step-1: Import required libraries

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
print("Aamir Shaikh 46")
print("==========================================")
print("All libraries are imported !!!")
```

```
Aamir Shaikh 46
==========================================
All libraries are imported !!!
```

Step-2: Read Dataset

```
data = {
'Amount': [100, 500, 1200, 50, 2500, 300, 700,20, 4000,150],
'Time': [10, 50, 60, 5, 120, 40, 90, 3, 200, 20],
'Fraud': [0, 0, 1, 0, 1, 0, 0, 0, 1, 0,]
}
print("Aamir Shaikh 46")
print("==========================================")
print("Dataset :")
print(data)
```

```
Aamir Shaikh 46
==========================================
Dataset :
{'Amount': [100, 500, 1200, 50, 2500, 300, 700, 20, 4000, 150],
'Time': [10, 50, 60, 5, 120, 40, 90, 3, 200, 20], 'Fraud': [0, 0, 1,
0, 1, 0, 0, 0, 1, 0]}
```

Step-3: Convert Dataset into Dataframe

```
df = pd.DataFrame(data)
print("Aamir Shaikh 46")
print("==========================================")
print("Dataframe:")
print(df)
```

```
Aamir Shaikh 46
==========================================
Dataframe:
```

```
   Amount  Time  Fraud
0     100    10      0
1     500    50      0
2    1200    60      1
3      50     5      0
4    2500   120      1
5     300    40      0
6     700    90      0
7      20     3      0
8    4000   200      1
9     150    20      0
```

Step-4: Divide dataset into features and target

```
X = df[['Amount', 'Time']] # features
y = df['Fraud']
print("Aamir Shaikh 46")
print("==========================================")
print("Features:")
print(X)
print("Target:")
print(y)
```

```
Aamir Shaikh 46
==========================================
Features:
   Amount  Time
0     100    10
1     500    50
2    1200    60
3      50     5
4    2500   120
5     300    40
6     700    90
7      20     3
8    4000   200
9     150    20
Target:
0    0
1    0
2    1
3    0
4    1
5    0
6    0
7    0
8    1
9    0
Name: Fraud, dtype: int64
```

Step-5: Divide dataset training and testing

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
print("Aamir Shaikh 46")
print("==========================================")
print("X_train :")
print(X_train)
print("X_test :")
print(X_test)
print("y_train")
print(y_train)
print("y_test :")
print(y_test)
```

```
Aamir Shaikh 45
==========================================
X_train :
   Amount  Time
5     300    40
0     100    10
7      20     3
2    1200    60
9     150    20
4    2500   120
3      50     5
6     700    90
X_test :
   Amount  Time
8    4000   200
1     500    50
y_train
5    0
0    0
7    0
2    1
9    0
4    1
3    0
6    0
Name: Fraud, dtype: int64
y_test :
8    1
1    0
Name: Fraud, dtype: int64
```

Step-6: Model Creation

```
model = LogisticRegression()
print("Aamir Shaikh 46")
print("==========================================")
print("Model Created !!!")
print(model)
```

```
Aamir Shaikh 46
==========================================
Model Created !!!
LogisticRegression()
```

Step-7: Model Training

```
model.fit(X_train, y_train)
print("Aamir Shaikh 46")
print("==========================================")
print("Model Trained !!!")
```

```
Aamir Shaikh 46
==========================================
Model Trained !!!
```

Step-8: Test Model

```
#test_data = [[500,20]]
y_pred = model.predict(X_test)
print("Aamir Shaikh 46")
print("==========================================")
print("Prediction :")
print(y_pred)
```

```
Aamir Shaikh 46
==========================================
Prediction :
[1 0]
```

Step-9: Find Accuracy of Model

```
print("Aamir Shaikh 46")
print("==========================================")
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Aamir Shaikh 46
==========================================
Accuracy: 1.0
```

Practical 3-A: Implementing a K-Nearest Neighbor (KNN) algorithm (e.g. Student Pass/Fail based on Attendance,Study Hours and Previous subjects marks)

Step-1: Import Required Libraries

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

print("Aamir Shaikh 46")
print("==========================================")
print("All libraries are imported !!!")
```

```
Aamir Shaikh 46
==========================================
All libraries are imported !!!
```

Step-2: Read Dataset

```
data = {
    "Study_Hours": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "Attendance": [40, 45, 50, 55, 60, 65, 70, 75, 80, 90],
    "Previous_Marks": [35, 40, 45, 50, 55, 60, 65, 70, 75, 85],
    "Result": ["Fail", "Fail","Fail", "Fail",
               "Pass", "Pass", "Pass", "Pass", "Pass", "Pass"] #
Corrected to 10 elements
}
print("Ayaa Aamir Shaikh 46")
print("==========================================")
print("Dataset :")
print(data)
```

```
Aamir Shaikh 46
==========================================
Dataset :
{'Study_Hours': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 'Attendance': [40,
45, 50, 55, 60, 65, 70, 75, 80, 90], 'Previous_Marks': [35, 40, 45,
50, 55, 60, 65, 70, 75, 85], 'Result': ['Fail', 'Fail', 'Fail',
'Fail', 'Pass', 'Pass', 'Pass', 'Pass', 'Pass', 'Pass']}
```

Step-3: Convert Dataset into Dataframe

```
df = pd.DataFrame(data)
print("Aamir Shiakh 46")
print("==========================================")
print(df)
```

```
Aamir Shaikh 46
==========================================
   Study_Hours  Attendance  Previous_Marks Result
0            1          40              35   Fail
1            2          45              40   Fail
2            3          50              45   Fail
3            4          55              50   Fail
4            5          60              55   Pass
5            6          65              60   Pass
6            7          70              65   Pass
7            8          75              70   Pass
8            9          80              75   Pass
9           10          90              85   Pass
```

Step-4: Divide dataset into features and target

```
X = df[["Study_Hours", "Attendance", "Previous_Marks"]]      # Input features
y = df["Result"]
print("Aamir Shaikh 46")
print("==========================================")
print("Features :")
print(X)
print("Target :")
print(y)
```

```
Aamir Shaikh 46
==========================================
Features :
   Study_Hours  Attendance  Previous_Marks
0            1          40              35
1            2          45              40
2            3          50              45
3            4          55              50
4            5          60              55
5            6          65              60
6            7          70              65
7            8          75              70
8            9          80              75
9           10          90              85
Target :
0    Fail
1    Fail
2    Fail
3    Fail
4    Pass
5    Pass
6    Pass
7    Pass
```

```
8     Pass
9     Pass
Name: Result, dtype: object
```

Step-5: Divide dataset into training and testing

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
print("Aamir Shaikh 46")
print("==========================================")
print("Training Data :")
print(X_train)
print(y_train)
print("Testing Data :")
print(X_test)
print(y_test)
```

```
Aamir Shaikh 46
==========================================
Training Data :
   Study_Hours  Attendance  Previous_Marks
5            6          65              60
0            1          40              35
7            8          75              70
2            3          50              45
9           10          90              85
4            5          60              55
3            4          55              50
6            7          70              65
5     Pass
0     Fail
7     Pass
2     Fail
9     Pass
4     Pass
3     Fail
6     Pass
Name: Result, dtype: object
Testing Data :
   Study Hours  Attendance  Previous Marks
8            9          80              75
1            2          45              40
8     Pass
1     Fail
Name: Result, dtype: object
```

Step-6: Model Creation

```
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)
print("Aamir Shaikh 46")
print("==========================================")
print("Model :")
print(model)
```

```
Aamir Shaikh 46
==========================================
Model :
KNeighborsClassifier(n_neighbors=3)
```

Step-7: Train Model

```
model.fit(X_train, y_train)
print("Aamir Shaikh 46")
print("==========================================")

print("Model Trained !!!")
```

```
Aamir Shaikh 46
==========================================
Model Trained !!!
```

Step-8: Test Model

```
y_pred = model.predict(X_test)
print("Aamir Shaikh 46")
print("==========================================")

print("Predicted Values :")
print(y_pred)
```

```
Ayaan Shaikh 45
==========================================
Predicted Values :
['Pass' 'Fail']
```

Step-9: Find accuracy of model

```
print("Aamir Shaikh 46")
print("==========================================")
print("Accuracy of Model :")
print(accuracy_score(y_test, y_pred) *100)
```

```
Aamir Shaikh 46
==========================================
Accuracy of Model :
100.0
```

Implementing a K-Nearest Neighbor (KNN) algorithm (e.g. to classify handwritten digits).

Step-1: Read Dataset

```
import pandas as pd
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

print("Aamir shaikh 46")
print("==========================================")
print("All libraries are imported !!!")
```

```
Aamir Shaikh 46
==========================================
All libraries are imported !!!
```

Step-2: Read Dataset

```
digits = load_digits()

print("Aamir shaikh 46")
print("==========================================")
print("Dataset:")
print(digits)
```

```
Aamir Shaikh 46
==========================================
Dataset:
{'data': array([[ 0.,  0.,  5., ...,  0.,  0.,  0.],
       [ 0.,  0.,  0., ..., 10.,  0.,  0.],
       [ 0.,  0.,  0., ..., 16.,  9.,  0.],
       ...,
       [ 0.,  0.,  1., ...,  6.,  0.,  0.],
       [ 0.,  0.,  2., ..., 12.,  0.,  0.],
       [ 0.,  0., 10., ..., 12.,  1.,  0.]]), 'target': array([0, 1,
2, ..., 8, 9, 8]), 'frame': None, 'feature_names': ['pixel_0_0',
'pixel_0_1', 'pixel_0_2', 'pixel_0_3', 'pixel_0_4', 'pixel_0_5',
'pixel_0_6', 'pixel_0_7', 'pixel_1_0', 'pixel_1_1', 'pixel_1_2',
'pixel_1_3', 'pixel_1_4', 'pixel_1_5', 'pixel_1_6', 'pixel_1_7',
'pixel_2_0', 'pixel_2_1', 'pixel_2_2', 'pixel_2_3', 'pixel_2_4',
'pixel_2_5', 'pixel_2_6', 'pixel_2_7', 'pixel_3_0', 'pixel_3_1',
'pixel_3_2', 'pixel_3_3', 'pixel_3_4', 'pixel_3_5', 'pixel_3_6',
'pixel_3_7', 'pixel_4_0', 'pixel_4_1', 'pixel_4_2', 'pixel_4_3',
'pixel_4_4', 'pixel_4_5', 'pixel_4_6', 'pixel_4_7', 'pixel_5_0',
'pixel_5_1', 'pixel_5_2', 'pixel_5_3', 'pixel_5_4', 'pixel_5_5',
'pixel_5_6', 'pixel_5_7', 'pixel_6_0', 'pixel_6_1', 'pixel_6_2',
```

```
'pixel_6_3', 'pixel_6_4', 'pixel_6_5', 'pixel_6_6', 'pixel_6_7',
'pixel_7_0', 'pixel_7_1', 'pixel_7_2', 'pixel_7_3', 'pixel_7_4',
'pixel_7_5', 'pixel_7_6', 'pixel_7_7'], 'target_names': array([0, 1,
2, 3, 4, 5, 6, 7, 8, 9]), 'images': array([[[ 0.,  0.,  5., ...,  1.,
0.,  0.],
        [ 0.,  0., 13., ..., 15.,  5.,  0.],
        [ 0.,  3., 15., ..., 11.,  8.,  0.],
        ...,
        [ 0.,  4., 11., ..., 12.,  7.,  0.],
        [ 0.,  2., 14., ..., 12.,  0.,  0.],
        [ 0.,  0.,  6., ...,  0.,  0.,  0.]],

       [[ 0.,  0.,  0., ...,  5.,  0.,  0.],
        [ 0.,  0.,  0., ...,  9.,  0.,  0.],
        [ 0.,  0.,  3., ...,  6.,  0.,  0.],
        ...,
        [ 0.,  0.,  1., ...,  6.,  0.,  0.],
        [ 0.,  0.,  1., ...,  6.,  0.,  0.],
        [ 0.,  0.,  0., ..., 10.,  0.,  0.]],

       [[ 0.,  0.,  0., ..., 12.,  0.,  0.],
        [ 0.,  0.,  3., ..., 14.,  0.,  0.],
        [ 0.,  0.,  8., ..., 16.,  0.,  0.],
        ...,
        [ 0.,  9., 16., ...,  0.,  0.,  0.],
        [ 0.,  3., 13., ..., 11.,  5.,  0.],
        [ 0.,  0.,  0., ..., 16.,  9.,  0.]],

       ...,

       [[ 0.,  0.,  1., ...,  1.,  0.,  0.],
        [ 0.,  0., 13., ...,  2.,  1.,  0.],
        [ 0.,  0., 16., ..., 16.,  5.,  0.],
        ...,
        [ 0.,  0., 16., ..., 15.,  0.,  0.],
        [ 0.,  0., 15., ..., 16.,  0.,  0.],
        [ 0.,  0.,  2., ...,  6.,  0.,  0.]],

       [[ 0.,  0.,  2., ...,  0.,  0.,  0.],
        [ 0.,  0., 14., ..., 15.,  1.,  0.],
        [ 0.,  4., 16., ..., 16.,  7.,  0.],
        ...,
        [ 0.,  0.,  0., ..., 16.,  2.,  0.],
        [ 0.,  0.,  4., ..., 16.,  2.,  0.],
        [ 0.,  0.,  5., ..., 12.,  0.,  0.]],

       [[ 0.,  0., 10., ...,  1.,  0.,  0.],
        [ 0.,  2., 16., ...,  1.,  0.,  0.],
        [ 0.,  0., 15., ..., 15.,  0.,  0.],
        ...,
```

```
        [ 0.,  4., 16., ..., 16.,  6.,  0.],
        [ 0.,  8., 16., ..., 16.,  8.,  0.],
        [ 0.,  1.,  8., ..., 12.,  1.,  0.]]]), 'DESCR': ".. 
_digits_dataset:\n\nOptical recognition of handwritten digits dataset\
n--------------------------------------------------\n\n**Data Set 
Characteristics:**\n\n:Number of Instances: 1797\n:Number of 
Attributes: 64\n:Attribute Information: 8x8 image of integer pixels in 
the range 0..16.\n:Missing Attribute Values: None\n:Creator: E. 
Alpaydin (alpaydin '@' boun.edu.tr)\n:Date: July; 1998\n\nThis is a 
copy of the test set of the UCI ML hand-written digits datasets\
nhttps://archive.ics.uci.edu/ml/datasets/Optical+Recognition+of+Handwr
itten+Digits\n\nThe data set contains images of hand-written digits: 
10 classes where\neach class refers to a digit.\n\nPreprocessing 
programs made available by NIST were used to extract\nnormalized 
bitmaps of handwritten digits from a preprinted form. From a\ntotal of 
43 people, 30 contributed to the training set and different 13\nto the 
test set. 32x32 bitmaps are divided into nonoverlapping blocks of\n4x4 
and the number of on pixels are counted in each block. This generates\
nan input matrix of 8x8 where each element is an integer in the range\
n0..16. This reduces dimensionality and gives invariance to small\
ndistortions.\n\nFor info on NIST preprocessing routines, see M. D. 
Garris, J. L. Blue, G.\nT. Candela, D. L. Dimmick, J. Geist, P. J. 
Grother, S. A. Janet, and C.\nL. Wilson, NIST Form-Based Handprint 
Recognition System, NISTIR 5469,\n1994.\n\n.. dropdown:: References\n\
n  - C. Kaynak (1995) Methods of Combining Multiple Classifiers and 
Their\n    Applications to Handwritten Digit Recognition, MSc Thesis, 
Institute of\n    Graduate Studies in Science and Engineering, 
Bogazici University.\n  - E. Alpaydin, C. Kaynak (1998) Cascading 
Classifiers, Kybernetika.\n  - Ken Tang and Ponnuthurai N. Suganthan 
and Xi Yao and A. Kai Qin.\n    Linear dimensionalityreduction using 
relevance weighted LDA. School of\n    Electrical and Electronic 
Engineering Nanyang Technological University.\n    2005.\n  - Claudio 
Gentile. A New Approximate Maximal Margin Classification\n 
Algorithm. NIPS. 2000.\n"}
```

Step-3: Divide dataset into features and target

```
X = digits.data                # Pixel values (8x8 images flattened to 
64 features)
y = digits.target              # Digit labels (0 to 9)

print("Ayaan Shaikh 45")
print("==========================================")
print("Features :")
print(y)
```

```
Aamir Shaikh 46
==========================================
```

```
Features :
[0 1 2 ... 8 9 8]
```

Step-4: Convert Dataset into DataFrame

```
df = pd.DataFrame(X)
print("Aamir shaikh 46")
print("==========================================")
print("DataFrame :")
print(df)
print("Dataset Shape:", df.shape)
```

```
Aamir Shaikh 46
==========================================
DataFrame :
       0    1     2     3     4     5    6    7    8    9   ...   54
55  \
0      0.0  0.0   5.0  13.0   9.0   1.0  0.0  0.0  0.0  0.0  ...  0.0
0.0
1      0.0  0.0   0.0  12.0  13.0   5.0  0.0  0.0  0.0  0.0  ...  0.0
0.0
2      0.0  0.0   0.0   4.0  15.0  12.0  0.0  0.0  0.0  0.0  ...  5.0
0.0
3      0.0  0.0   7.0  15.0  13.0   1.0  0.0  0.0  0.0  8.0  ...  9.0
0.0
4      0.0  0.0   0.0   1.0  11.0   0.0  0.0  0.0  0.0  0.0  ...  0.0
0.0
...    ...  ...   ...   ...   ...   ...  ...  ...  ...  ...  ...  ...
...
1792   0.0  0.0   4.0  10.0  13.0   6.0  0.0  0.0  0.0  1.0  ...  4.0
0.0
1793   0.0  0.0   6.0  16.0  13.0  11.0  1.0  0.0  0.0  0.0  ...  1.0
0.0
1794   0.0  0.0   1.0  11.0  15.0   1.0  0.0  0.0  0.0  0.0  ...  0.0
0.0
1795   0.0  0.0   2.0  10.0   7.0   0.0  0.0  0.0  0.0  0.0  ...  2.0
0.0
1796   0.0  0.0  10.0  14.0   8.0   1.0  0.0  0.0  0.0  2.0  ...  8.0
0.0

        56   57   58    59    60    61   62   63
0      0.0  0.0  6.0  13.0  10.0   0.0  0.0  0.0
1      0.0  0.0  0.0  11.0  16.0  10.0  0.0  0.0
2      0.0  0.0  0.0   3.0  11.0  16.0  9.0  0.0
3      0.0  0.0  7.0  13.0  13.0   9.0  0.0  0.0
4      0.0  0.0  0.0   2.0  16.0   4.0  0.0  0.0
...    ...  ...  ...   ...   ...   ...  ...  ...
1792   0.0  0.0  2.0  14.0  15.0   9.0  0.0  0.0
1793   0.0  0.0  6.0  16.0  14.0   6.0  0.0  0.0
```

```
1794   0.0   0.0   2.0    9.0   13.0    6.0   0.0   0.0
1795   0.0   0.0   5.0   12.0   16.0   12.0   0.0   0.0
1796   0.0   1.0   8.0   12.0   14.0   12.0   1.0   0.0

[1797 rows x 64 columns]
Dataset Shape: (1797, 64)
```

Step-5: TRAIN-TEST SPLI

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
print("Aamir shaikh 46")
print("==========================================")
print("Train-Test Split")
print("X_train:")
print(X_train)
print("X_test:")
print(X_test)
print("y_train:")
print(y_train)
print("y_test:")
print(y_test)
```

```
Aamir Shaikh 46
==========================================
Train-Test Split
X_train:
[[ 0.   0.   3. ... 13.   4.   0.]
 [ 0.   0.   9. ...  3.   0.   0.]
 [ 0.   0.   0. ...  6.   0.   0.]
 ...
 [ 0.   0.   9. ... 16.   2.   0.]
 [ 0.   0.   1. ...  0.   0.   0.]
 [ 0.   0.   1. ...  1.   0.   0.]]
X_test:
[[ 0.   0.   0. ... 14.   5.   0.]
 [ 0.   0.  11. ...  1.   0.   0.]
 [ 0.   0.   8. ...  8.   0.   0.]
 ...
 [ 0.   0.   7. ... 10.   0.   0.]
 [ 0.   0.   7. ... 16.   4.   0.]
 [ 0.   0.  14. ...  5.   0.   0.]]
y_train:
[6 0 0 ... 2 7 1]
y_test:
[6 9 3 7 2 1 5 2 5 2 1 9 4 0 4 2 3 7 8 8 4 3 9 7 5 6 3 5 6 3 4 9 1 4 4
6 9
 4 7 6 6 9 1 3 6 1 3 0 6 5 5 1 9 5 6 0 9 0 0 1 0 4 5 2 4 5 7 0 7 5 9 5
5 4
```

```
 7 0 4 5 5 9 9 0 2 3 8 0 6 4 4 9 1 2 8 3 5 2 9 0 4 4 4 3 5 3 1 3 5 9 4
2 7
 7 4 4 1 9 2 7 8 7 2 6 9 4 0 7 2 7 5 8 7 5 7 7 0 6 6 4 2 8 0 9 4 6 9 9
6 9
 0 3 5 6 6 0 6 4 3 9 3 9 7 2 9 0 4 5 3 6 5 9 9 8 4 2 1 3 7 7 2 2 3 9 8
0 3
 2 2 5 6 9 9 4 1 5 4 2 3 6 4 8 5 9 5 7 8 9 4 8 1 5 4 4 9 6 1 8 6 0 4 5
2 7
 4 6 4 5 6 0 3 2 3 6 7 1 5 1 4 7 6 8 8 5 5 1 6 2 8 8 9 9 7 6 2 2 2 3 4
8 8
 3 6 0 9 7 7 0 1 0 4 5 1 5 3 6 0 4 1 0 0 3 6 5 9 7 3 5 5 9 9 8 5 3 3 2
0 5
 8 3 4 0 2 4 6 4 3 4 5 0 5 2 1 3 1 4 1 1 7 0 1 5 2 1 2 8 7 0 6 4 8 8 5
1 8
 4 5 8 7 9 8 5 0 6 2 0 7 9 8 9 5 2 7 7 1 8 7 4 3 8 3 5]
```

Step-6: Create KNN Model

```
model = KNeighborsClassifier(n_neighbors=5)
print("Ayaan Shaikh 45")
print("==========================================")
print("Model is created !!!")
print(model)
```

```
Aamir Shaikh 46
==========================================
Model is created !!!
KNeighborsClassifier()
```

Step-7: Model training

```
model.fit(X_train, y_train)
print("Aamir shaikh 46")
print("==========================================")
print("Model is trained !!!")
```

```
Aamir Shaikh 46
==========================================
Model is trained !!!
```

Step-8: Predict on Test set

```
y_pred = model.predict(X_test)
print("Aamir shaikh 46")
print("==========================================")
print("Predicted Values :")
print(y_pred)
```

```
Aamir Shaikh 46
==========================================
Predicted Values :
[6 9 3 7 2 1 5 2 5 2 1 9 4 0 4 2 3 7 8 8 4 3 9 7 5 6 3 5 6 3 4 9 1 4 4
6 9
 4 7 6 6 9 1 3 6 1 3 0 6 5 5 1 9 5 6 0 9 0 0 1 0 4 5 2 4 5 7 0 7 5 9 9
5 4
 7 0 4 5 5 9 9 0 2 3 8 0 6 4 4 9 1 2 8 3 5 2 9 0 4 4 4 3 5 3 1 3 5 9 4
2 7
 7 4 4 1 9 2 7 8 7 2 6 9 4 0 7 2 7 5 8 7 5 7 9 0 6 6 4 2 8 0 9 4 6 9 9
6 9
 0 3 5 6 6 0 6 4 3 9 3 4 7 2 9 0 4 5 3 6 5 9 9 8 4 2 1 3 7 7 2 2 3 9 8
0 3
 2 2 5 6 9 9 4 1 5 4 2 3 6 4 8 5 9 5 7 8 9 4 8 1 5 4 4 9 6 1 8 6 0 4 5
2 7
 4 6 4 5 6 0 3 2 3 6 7 1 5 1 4 7 6 8 8 5 5 1 6 2 8 8 9 5 7 6 2 2 2 3 4
8 8
 3 6 0 9 7 7 0 1 0 4 5 1 5 3 6 0 4 1 0 0 3 6 5 9 7 3 5 5 9 9 8 5 3 3 2
0 5
 8 3 4 0 2 4 6 4 3 4 5 0 5 2 1 3 1 4 1 1 7 0 1 5 2 1 2 8 7 0 6 4 8 8 5
1 8
 4 5 8 7 9 8 6 0 6 2 0 7 9 8 9 5 2 7 7 1 8 7 4 3 8 3 5]
```

Step-9: Evaluation

```python
print("Aamir shaikh 46")
print("==========================================")
print("Accuracy:", accuracy_score(y_test, y_pred)*100)
```

```
Aamir Shaikh 46
==========================================
Accuracy: 98.61111111111111
```

Practical No.4 (A): Implementation of Decision Tree Classifier with iris dataset

---

Step-1: Import Required Libraries

```
import pandas as pd
print("Aamir Shaikh 46")
print("==========================================")
print("All libraries are imported !!!")
data = pd.read_csv('/content/iris.csv')
print(data)
```

```
Aamir Shaikh 46
==========================================
All libraries are imported !!!
     sepal_length  sepal_width  petal_length  petal_width    species
0             5.1          3.5           1.4          0.2     setosa
1             4.9          3.0           1.4          0.2     setosa
2             4.7          3.2           1.3          0.2     setosa
3             4.6          3.1           1.5          0.2     setosa
4             5.0          3.6           1.4          0.2     setosa
..            ...          ...           ...          ...        ...
145           6.7          3.0           5.2          2.3  virginica
146           6.3          2.5           5.0          1.9  virginica
147           6.5          3.0           5.2          2.0  virginica
148           6.2          3.4           5.4          2.3  virginica
149           5.9          3.0           5.1          1.8  virginica

[150 rows x 5 columns]
```

```
from sklearn.datasets import load_iris
print("Aamir Shaikh 46")
print("==========================================")
iris = load_iris()
print(iris)
```

```
Aamir Shaikh 46
==========================================
{'data': array([[5.1, 3.5, 1.4, 0.2],
       [4.9, 3. , 1.4, 0.2],
       [4.7, 3.2, 1.3, 0.2],
       [4.6, 3.1, 1.5, 0.2],
       [5. , 3.6, 1.4, 0.2],
       [5.4, 3.9, 1.7, 0.4],
       [4.6, 3.4, 1.4, 0.3],
       [5. , 3.4, 1.5, 0.2],
       [4.4, 2.9, 1.4, 0.2],
       [4.9, 3.1, 1.5, 0.1],
```

```
       [5.4, 3.7, 1.5, 0.2],
       [4.8, 3.4, 1.6, 0.2],
       [4.8, 3. , 1.4, 0.1],
       [4.3, 3. , 1.1, 0.1],
       [5.8, 4. , 1.2, 0.2],
       [5.7, 4.4, 1.5, 0.4],
       [5.4, 3.9, 1.3, 0.4],
       [5.1, 3.5, 1.4, 0.3],
       [5.7, 3.8, 1.7, 0.3],
       [5.1, 3.8, 1.5, 0.3],
       [5.4, 3.4, 1.7, 0.2],
       [5.1, 3.7, 1.5, 0.4],
       [4.6, 3.6, 1. , 0.2],
       [5.1, 3.3, 1.7, 0.5],
       [4.8, 3.4, 1.9, 0.2],
       [5. , 3. , 1.6, 0.2],
       [5. , 3.4, 1.6, 0.4],
       [5.2, 3.5, 1.5, 0.2],
       [5.2, 3.4, 1.4, 0.2],
       [4.7, 3.2, 1.6, 0.2],
       [4.8, 3.1, 1.6, 0.2],
       [5.4, 3.4, 1.5, 0.4],
       [5.2, 4.1, 1.5, 0.1],
       [5.5, 4.2, 1.4, 0.2],
       [4.9, 3.1, 1.5, 0.2],
       [5. , 3.2, 1.2, 0.2],
       [5.5, 3.5, 1.3, 0.2],
       [4.9, 3.6, 1.4, 0.1],
       [4.4, 3. , 1.3, 0.2],
       [5.1, 3.4, 1.5, 0.2],
       [5. , 3.5, 1.3, 0.3],
       [4.5, 2.3, 1.3, 0.3],
       [4.4, 3.2, 1.3, 0.2],
       [5. , 3.5, 1.6, 0.6],
       [5.1, 3.8, 1.9, 0.4],
       [4.8, 3. , 1.4, 0.3],
       [5.1, 3.8, 1.6, 0.2],
       [4.6, 3.2, 1.4, 0.2],
       [5.3, 3.7, 1.5, 0.2],
       [5. , 3.3, 1.4, 0.2],
       [7. , 3.2, 4.7, 1.4],
       [6.4, 3.2, 4.5, 1.5],
       [6.9, 3.1, 4.9, 1.5],
       [5.5, 2.3, 4. , 1.3],
       [6.5, 2.8, 4.6, 1.5],
       [5.7, 2.8, 4.5, 1.3],
       [6.3, 3.3, 4.7, 1.6],
       [4.9, 2.4, 3.3, 1. ],
       [6.6, 2.9, 4.6, 1.3],
```

```
       [5.2, 2.7, 3.9, 1.4],
       [5. , 2. , 3.5, 1. ],
       [5.9, 3. , 4.2, 1.5],
       [6. , 2.2, 4. , 1. ],
       [6.1, 2.9, 4.7, 1.4],
       [5.6, 2.9, 3.6, 1.3],
       [6.7, 3.1, 4.4, 1.4],
       [5.6, 3. , 4.5, 1.5],
       [5.8, 2.7, 4.1, 1. ],
       [6.2, 2.2, 4.5, 1.5],
       [5.6, 2.5, 3.9, 1.1],
       [5.9, 3.2, 4.8, 1.8],
       [6.1, 2.8, 4. , 1.3],
       [6.3, 2.5, 4.9, 1.5],
       [6.1, 2.8, 4.7, 1.2],
       [6.4, 2.9, 4.3, 1.3],
       [6.6, 3. , 4.4, 1.4],
       [6.8, 2.8, 4.8, 1.4],
       [6.7, 3. , 5. , 1.7],
       [6. , 2.9, 4.5, 1.5],
       [5.7, 2.6, 3.5, 1. ],
       [5.5, 2.4, 3.8, 1.1],
       [5.5, 2.4, 3.7, 1. ],
       [5.8, 2.7, 3.9, 1.2],
       [6. , 2.7, 5.1, 1.6],
       [5.4, 3. , 4.5, 1.5],
       [6. , 3.4, 4.5, 1.6],
       [6.7, 3.1, 4.7, 1.5],
       [6.3, 2.3, 4.4, 1.3],
       [5.6, 3. , 4.1, 1.3],
       [5.5, 2.5, 4. , 1.3],
       [5.5, 2.6, 4.4, 1.2],
       [6.1, 3. , 4.6, 1.4],
       [5.8, 2.6, 4. , 1.2],
       [5. , 2.3, 3.3, 1. ],
       [5.6, 2.7, 4.2, 1.3],
       [5.7, 3. , 4.2, 1.2],
       [5.7, 2.9, 4.2, 1.3],
       [6.2, 2.9, 4.3, 1.3],
       [5.1, 2.5, 3. , 1.1],
       [5.7, 2.8, 4.1, 1.3],
       [6.3, 3.3, 6. , 2.5],
       [5.8, 2.7, 5.1, 1.9],
       [7.1, 3. , 5.9, 2.1],
       [6.3, 2.9, 5.6, 1.8],
       [6.5, 3. , 5.8, 2.2],
       [7.6, 3. , 6.6, 2.1],
       [4.9, 2.5, 4.5, 1.7],
       [7.3, 2.9, 6.3, 1.8],
```

```
        [6.7, 2.5, 5.8, 1.8],
        [7.2, 3.6, 6.1, 2.5],
        [6.5, 3.2, 5.1, 2. ],
        [6.4, 2.7, 5.3, 1.9],
        [6.8, 3. , 5.5, 2.1],
        [5.7, 2.5, 5. , 2. ],
        [5.8, 2.8, 5.1, 2.4],
        [6.4, 3.2, 5.3, 2.3],
        [6.5, 3. , 5.5, 1.8],
        [7.7, 3.8, 6.7, 2.2],
        [7.7, 2.6, 6.9, 2.3],
        [6. , 2.2, 5. , 1.5],
        [6.9, 3.2, 5.7, 2.3],
        [5.6, 2.8, 4.9, 2. ],
        [7.7, 2.8, 6.7, 2. ],
        [6.3, 2.7, 4.9, 1.8],
        [6.7, 3.3, 5.7, 2.1],
        [7.2, 3.2, 6. , 1.8],
        [6.2, 2.8, 4.8, 1.8],
        [6.1, 3. , 4.9, 1.8],
        [6.4, 2.8, 5.6, 2.1],
        [7.2, 3. , 5.8, 1.6],
        [7.4, 2.8, 6.1, 1.9],
        [7.9, 3.8, 6.4, 2. ],
        [6.4, 2.8, 5.6, 2.2],
        [6.3, 2.8, 5.1, 1.5],
        [6.1, 2.6, 5.6, 1.4],
        [7.7, 3. , 6.1, 2.3],
        [6.3, 3.4, 5.6, 2.4],
        [6.4, 3.1, 5.5, 1.8],
        [6. , 3. , 4.8, 1.8],
        [6.9, 3.1, 5.4, 2.1],
        [6.7, 3.1, 5.6, 2.4],
        [6.9, 3.1, 5.1, 2.3],
        [5.8, 2.7, 5.1, 1.9],
        [6.8, 3.2, 5.9, 2.3],
        [6.7, 3.3, 5.7, 2.5],
        [6.7, 3. , 5.2, 2.3],
        [6.3, 2.5, 5. , 1.9],
        [6.5, 3. , 5.2, 2. ],
        [6.2, 3.4, 5.4, 2.3],
        [5.9, 3. , 5.1, 1.8]]), 'target': array([0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,
        0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1,
```

```
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]),
'frame': None, 'target_names': array(['setosa', 'versicolor',
'virginica'], dtype='<U10'), 'DESCR': '.. _iris_dataset:\n\nIris
plants dataset\n--------------------\n\n**Data Set Characteristics:**\
n\n:Number of Instances: 150 (50 in each of three classes)\n:Number of
Attributes: 4 numeric, predictive attributes and the class\n:Attribute
Information:\n    - sepal length in cm\n    - sepal width in cm\n    -
petal length in cm\n    - petal width in cm\n    - class:\n
- Iris-Setosa\n            - Iris-Versicolour\n            - Iris-
Virginica\n\n:Summary Statistics:\n\n============== ==== ==== =======
===== ====================\n                 Min  Max   Mean    SD
Class Correlation\n============== ==== ==== ======= =====
====================\nsepal length:   4.3  7.9   5.84   0.83
0.7826\nsepal width:    2.0  4.4   3.05   0.43   -0.4194\npetal
length:   1.0  6.9   3.76   1.76    0.9490  (high!)\npetal width:
0.1  2.5   1.20   0.76    0.9565  (high!)\n============== ==== ====
======= ===== ====================\n\n:Missing Attribute Values: None\
n:Class Distribution: 33.3% for each of 3 classes.\n:Creator: R.A.
Fisher\n:Donor: Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)\
n:Date: July, 1988\n\nThe famous Iris database, first used by Sir R.A.
Fisher. The dataset is taken\nfrom Fisher\'s paper. Note that it\'s
the same as in R, but not as in the UCI\nMachine Learning Repository,
which has two wrong data points.\n\nThis is perhaps the best known
database to be found in the\npattern recognition literature.
Fisher\'s paper is a classic in the field and\nis referenced
frequently to this day.  (See Duda & Hart, for example.)  The\ndata
set contains 3 classes of 50 instances each, where each class refers
to a\ntype of iris plant.  One class is linearly separable from the
other 2; the\nlatter are NOT linearly separable from each other.\n\n..
dropdown:: References\n\n  - Fisher, R.A. "The use of multiple
measurements in taxonomic problems"\n    Annual Eugenics, 7, Part II,
179-188 (1936); also in "Contributions to\n    Mathematical
Statistics" (John Wiley, NY, 1950).\n  - Duda, R.O., & Hart, P.E.
(1973) Pattern Classification and Scene Analysis.\n    (Q327.D83) John
Wiley & Sons.  ISBN 0-471-22361-1.  See page 218.\n  - Dasarathy, B.V.
(1980) "Nosing Around the Neighborhood: A New System\n    Structure
and Classification Rule for Recognition in Partially Exposed\n
Environments".  IEEE Transactions on Pattern Analysis and Machine\n
Intelligence, Vol. PAMI-2, No. 1, 67-71.\n  - Gates, G.W. (1972) "The
Reduced Nearest Neighbor Rule".  IEEE Transactions\n    on Information
Theory, May 1972, 431-433.\n  - See also: 1988 MLC Proceedings, 54-64.
Cheeseman et al"s AUTOCLASS II\n    conceptual clustering system finds
3 classes in the data.\n  - Many, many more ...\n', 'feature_names':
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal
```

```
width (cm)'], 'filename': 'iris.csv', 'data_module':
'sklearn.datasets.data'}
```

Step-2: Divide Dataset into Features(x) and Target(y)

```
import pandas as pd
print("Aamir Shaikh 46")
print("==========================================")
x = pd.DataFrame(iris.data, columns = iris.feature_names)
y = pd.Series(iris.target, name = "Target")
print("Features")
print(x)
print("Target")
print(y)
```

```
Aamir Shaikh 46
==========================================
Features
     sepal length (cm)  sepal width (cm)  petal length (cm)  petal
width (cm)
0                  5.1               3.5                1.4
0.2
1                  4.9               3.0                1.4
0.2
2                  4.7               3.2                1.3
0.2
3                  4.6               3.1                1.5
0.2
4                  5.0               3.6                1.4
0.2
..                 ...               ...                ...
...
145                6.7               3.0                5.2
2.3
146                6.3               2.5                5.0
1.9
147                6.5               3.0                5.2
2.0
148                6.2               3.4                5.4
2.3
149                5.9               3.0                5.1
1.8

[150 rows x 4 columns]
Target
0      0
1      0
```

```
2       0
3       0
4       0
       ..
145     2
146     2
147     2
148     2
149     2
Name: Target, Length: 150, dtype: int64
```

Step-3: Split data into training (80%) and testing (20%)

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(x, y,
test_size=0.2, random_state=42)

print("Aamir Shaikh 46")
print("==========================================")
print("X_train")
print(X_train)
print("X_test")
print(X_test)
print("Ayaan Shaikh 45")
print("==========================================")
```

```
Aamir Shaikh 46
==========================================
X_train
     sepal length (cm)  sepal width (cm)  petal length (cm)  petal
width (cm)
22                 4.6               3.6                1.0
0.2
15                 5.7               4.4                1.5
0.4
65                 6.7               3.1                4.4
1.4
11                 4.8               3.4                1.6
0.2
42                 4.4               3.2                1.3
0.2
..                 ...               ...                ...
...
71                 6.1               2.8                4.0
1.3
106                4.9               2.5                4.5
```

```
1.7
14                  5.8                 4.0                 1.2
0.2
92                  5.8                 2.6                 4.0
1.2
102                 7.1                 3.0                 5.9
2.1

[120 rows x 4 columns]
X_test
     sepal length (cm)  sepal width (cm)  petal length (cm)  petal
width (cm)
73                  6.1                 2.8                 4.7
1.2
18                  5.7                 3.8                 1.7
0.3
118                 7.7                 2.6                 6.9
2.3
78                  6.0                 2.9                 4.5
1.5
76                  6.8                 2.8                 4.8
1.4
31                  5.4                 3.4                 1.5
0.4
64                  5.6                 2.9                 3.6
1.3
141                 6.9                 3.1                 5.1
2.3
68                  6.2                 2.2                 4.5
1.5
82                  5.8                 2.7                 3.9
1.2
110                 6.5                 3.2                 5.1
2.0
12                  4.8                 3.0                 1.4
0.1
36                  5.5                 3.5                 1.3
0.2
9                   4.9                 3.1                 1.5
0.1
19                  5.1                 3.8                 1.5
0.3
56                  6.3                 3.3                 4.7
1.6
104                 6.5                 3.0                 5.8
2.2
69                  5.6                 2.5                 3.9
1.1
55                  5.7                 2.8                 4.5
```

```
1.3
132                 6.4                 2.8                 5.6
2.2
29                  4.7                 3.2                 1.6
0.2
127                 6.1                 3.0                 4.9
1.8
26                  5.0                 3.4                 1.6
0.4
128                 6.4                 2.8                 5.6
2.1
131                 7.9                 3.8                 6.4
2.0
145                 6.7                 3.0                 5.2
2.3
108                 6.7                 2.5                 5.8
1.8
143                 6.8                 3.2                 5.9
2.3
45                  4.8                 3.0                 1.4
0.3
30                  4.8                 3.1                 1.6
0.2
Ayaan Shaikh 45
==========================================
```

Step-4: Model Creation and Training

```
from sklearn.tree import DecisionTreeClassifier

#model DecisionTreeClassifier()
#model DecisionTreeClassifier(criterion='gini',
random_state=42,max_depth = 4)
model = DecisionTreeClassifier(criterion = 'entropy',
random_state=42,max_depth=4)

print("Aamir Shaikh 46")
print("==========================================")

model.fit(X_train, y_train)

print("Model is trained")

Aamir Shaikh 46
==========================================
Model is trained
```

Step-5: Test Model

```
test_data = [[6,3.4,4.5,1.6]]
y_pred = model.predict(X_test)
print("Aamir Shaikh 46")
print("==========================================")
print("Predicted values")
print(y_pred)
```

```
Aamir Shaikh 46
==========================================
Predicted values
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
```

Step-6: Evaluate Accuracy of Model

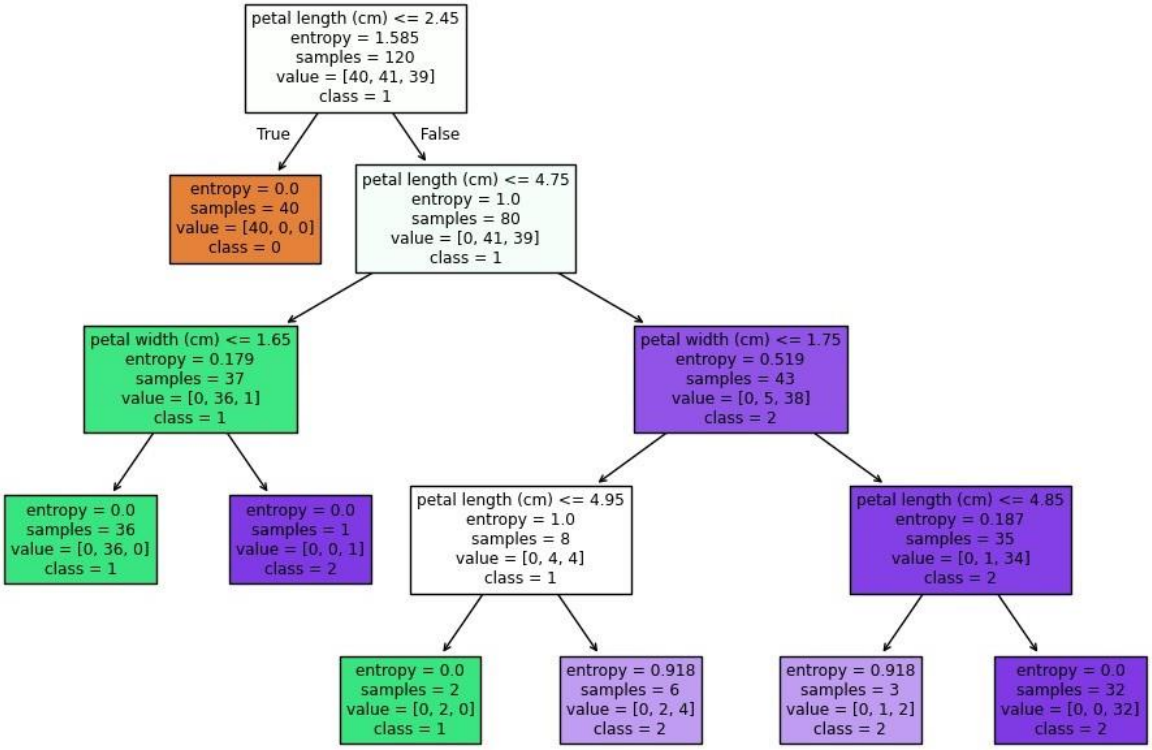
```
from sklearn.metrics import accuracy_score

accuracy  = accuracy_score(y_test, y_pred)
print("Aamir Shaikh 46")
print("==========================================")
print("Accuracy:", accuracy)
```

```
Aamir Shaikh 46
==========================================
Accuracy: 1.0
```

Step-7: Plot Decision Tree

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt
plt.figure(figsize=(12,8))
plot_tree(model, filled=True, feature_names=x.columns,
class_names=y.unique().astype(str))
plt.title("Decision Tree Visualization")
print("Aamir Shaikh 46")
print("==========================================")
plt.show()
Aamir Shaikh 46
==========================================
```

Decision Tree Visualization



Building a decision tree model using the ID3 algorithm (e.g. to predict whether a customer will churn or not) in python

---

Step-1: Import Required Libraries

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix
print("Aamir Shaikh 46")
print("==========================================")
print("All libraries are imported !!!")
```

```
Aamir Shaikh 46
==========================================
All libraries are imported !!!
```

Step-2: Create Dataset

```
data = {
    'Age': ['Young', 'Young', 'Middle', 'Old', 'Old', 'Middle',
'Young', 'Old'],
    'Income':
['High','High','High','Medium','Low','Medium','Medium','High'],
    'Contract': ['Monthly', 'Yearly', 'Monthly', 'Monthly', 'Yearly',
'Yearly', 'Monthly', 'Yearly'],
    'Churn': ['Yes','No','Yes','Yes','No','No','Yes','No']
}


print("Aamir Shaikh 46")
print("==========================================")
df = pd.DataFrame(data)
print("Original Dataset: \n", df)
```

```
Aamir Shaikh 46
==========================================
Original Dataset:
       Age  Income Contract Churn
0   Young    High  Monthly   Yes
1   Young    High   Yearly    No
2  Middle    High  Monthly   Yes
3     Old  Medium  Monthly   Yes
4     Old     Low   Yearly    No
5  Middle  Medium   Yearly    No
```

```
6   Young  Medium  Monthly   Yes
7     Old    High   Yearly    No
```

Step-3: Encode Categorical Data

```
le = LabelEncoder()

for col in df.columns:
  df[col] = le.fit_transform(df[col])
  print("Aamir Shaikh 46")
  print("==========================================")

  print("\nEncoded Dataset:\n", df)
```

```
Aamir Shaikh 46
==========================================

Encoded Dataset:
    Age  Income Contract Churn
0     2    High  Monthly   Yes
1     2    High   Yearly    No
2     0    High  Monthly   Yes
3     1  Medium  Monthly   Yes
4     1     Low   Yearly    No
5     0  Medium   Yearly    No
6     2  Medium  Monthly   Yes
7     1    High   Yearly    No

Aamir Shaikh 46
==========================================

Encoded Dataset:
    Age  Income Contract Churn
0     2       0  Monthly   Yes
1     2       0   Yearly    No
2     0       0  Monthly   Yes
3     1       2  Monthly   Yes
4     1       1   Yearly    No
5     0       2   Yearly    No
6     2       2  Monthly   Yes
7     1       0   Yearly    No

Aamir Shaikh 46
==========================================

Encoded Dataset:
    Age  Income  Contract Churn
0     2       0         0   Yes
```

```
1     2        0          1     No
2     0        0          0    Yes
3     1        2          0    Yes
4     1        1          1     No
5     0        2          1     No
6     2        2          0    Yes
7     1        0          1     No

Aamir Shaikh 46
==========================================

Encoded Dataset:
0     2        0          0       1
1     2        0          1       0
2     0        0          0       1
3     1        2          0       1
4     1        1          1       0
5     0        2          1       0
6     2        2          0       1
7     1        0          1       0
```

Step-4: Split Features and Target

```
X = df.drop('Churn', axis=1)
y = df['Churn']

print("Ayaan Shaikh 45")
print("==========================================")
print("Features: ")
print(X)
print("Target: ")
print(y)
```

```
Aamir Shaikh 46
==========================================
Features:
   Age  Income  Contract
0    2       0         0
1    2       0         1
2    0       0         0
3    1       2         0
4    1       1         1
5    0       2         1
6    2       2         0
7    1       0         1
Target:
0    1
```

```
1     0
2     1
```

```
3    1
4    0
5    0
6    1
7    0
Name: Churn, dtype: int64
```

Step-5: Train-Test Split

```
from sklearn.model_selection import train_test_split

print("Aamir Shaikh 46")
print("==========================================")
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.20, random_state=42)
print("X_train: \n", X_train)
print("X_test: \n", X_test)
print("y_train: \n", y_train)
print("y_test: \n", y_test)
```

```
Aamir Shaikh 46
==========================================
X_train:
    Age  Income  Contract
0    2       0         0
7    1       0         1
2    0       0         0
4    1       1         1
3    1       2         0
6    2       2         0
X_test:
    Age  Income  Contract
1    2       0         1
5    0       2         1
y_train:
 0    1
7    0
2    1
4    0
3    1
6    1
Name: Churn, dtype: int64
y_test:
 1    0
5    0
Name: Churn, dtype: int64
```

Step-6: Model Creation and Training

```
model = DecisionTreeClassifier(criterion='gini')
model.fit(X_train, y_train)

print("Aamir Shaikh 46")
print("==========================================")
print("Model Created and Trained !!!")
```

```
Aamir Shaikh 46
==========================================
Model Created and Trained !!!
```

Step-7: Prediction

```
y_pred = model.predict(X_test)

print("Aamir Shaikh 46")
print("==========================================")
print("Prediction: ", y_pred)
```

```
Aamir Shaikh 46
==========================================
Prediction:  [0 0]
```

Step-8: Evaluation

```
print("Aamir Shaikh 46")
print("==========================================")

print("\nAccuracy: ", accuracy_score(y_test, y_pred)*100)
```

```
Aamir Shaikh 46
==========================================

Accuracy:  100.0
```

Step-9: Predict for New Customer

```
# New Customer: Young, Medium Income, Monthly Contract
# (Use encoded values based on LabelEncoder output
new_customer = [[2, 2, 1]]

prediction = model.predict(new_customer)
```

```python
print("\nNew Customer Churn Prediction",
      "Yes" if prediction[0] == 1 else "No")


print("Aamir Shaikh 46")
print("==========================================")
```

```
New Customer Churn Prediction No

Aamir Shaikh 46
==========================================

/usr/local/lib/python3.12/dist-packages/sklearn/utils/
validation.py:2739: UserWarning: X does not have valid feature names,
but DecisionTreeClassifier was fitted with feature names
```

Practical No.4 (C): Implementation of Decision Tree Classifier using Gini Index

---

Step-1: Read Dataset

```
import pandas as pd

# Create sample Dataset

data = {
    'Age': ['Younge', 'Younge', 'Middle', 'Senior', 'Senior',
'Senior', 'Middle', 'Young'],
    'Income': ['High', 'High', 'High', 'Medium', 'Low', 'Low', 'Low',
'Medium'],
    'Student': ['No', 'No', 'No', 'No', 'Yes', 'Yes', 'Yes', 'No'],
    'Credit_Rating': ['Fair', 'Excellent', 'Fair', 'Fair', 'Fair',
'Excellent','Excellent', 'Fair'],
    'Buy': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No']

}

df = pd.DataFrame(data)
print("Aamir Shaikh 46")
print("==========================================")
print("All libraries are imported !!!")
print(df)
```

```
Aamir Shaikh 46
==========================================
All libraries are imported !!!
      Age  Income Student Credit_Rating  Buy
0  Younge    High      No          Fair   No
1  Younge    High      No     Excellent   No
2  Middle    High      No          Fair  Yes
3  Senior  Medium      No          Fair  Yes
4  Senior     Low     Yes          Fair  Yes
5  Senior     Low     Yes     Excellent   No
6  Middle     Low     Yes     Excellent  Yes
7   Young  Medium      No          Fair   No
```

Step-2: Convert Categorical Data to Numbers(Decision Tree Needs Numeric Data)

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()

for column in df.columns:
  df[column] = le.fit_transform(df[column])
```

```
print("Aamir Shaikh 46")
print("==========================================")
print(df)
```

```
Aamir Shaikh 46
==========================================
    Age  Income  Student  Credit_Rating  Buy
0     3       0        0              1    0
1     3       0        0              0    0
2     0       0        0              1    1
3     1       2        0              1    1
4     1       1        1              1    1
5     1       1        1              0    0
6     0       1        1              0    1
7     2       2        0              1    0
```

Step-3: Split Features and Target

```
X = df.drop('Buy', axis=1)
y = df['Buy']
print("Aamir Shaikh 46")
print("==========================================")
print("Features")
print(X)
print("Target:")
print(y)
```

```
Aamir Shaikh 46
==========================================
Features
   Age  Income  Student  Credit_Rating
0    3       0        0              1
1    3       0        0              0
2    0       0        0              1
3    1       2        0              1
4    1       1        1              1
5    1       1        1              0
6    0       1        1              0
7    2       2        0              1
Target:
0    0
1    0
2    1
3    1
4    1
5    0
6    1
```

```
7    0
Name: Buy, dtype: int64
```

Step-4: Train Decision Tree using Gini Index

```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(criterion='gini')
model.fit(X, y)

print("Aamir Shaikh 46")
print("==========================================")
print("Model is Trained !!!")
print(model)
```

```
Aamir Shaikh 46
==========================================
Model is Trained !!!
DecisionTreeClassifier()
```

Step-5: Make Prediction

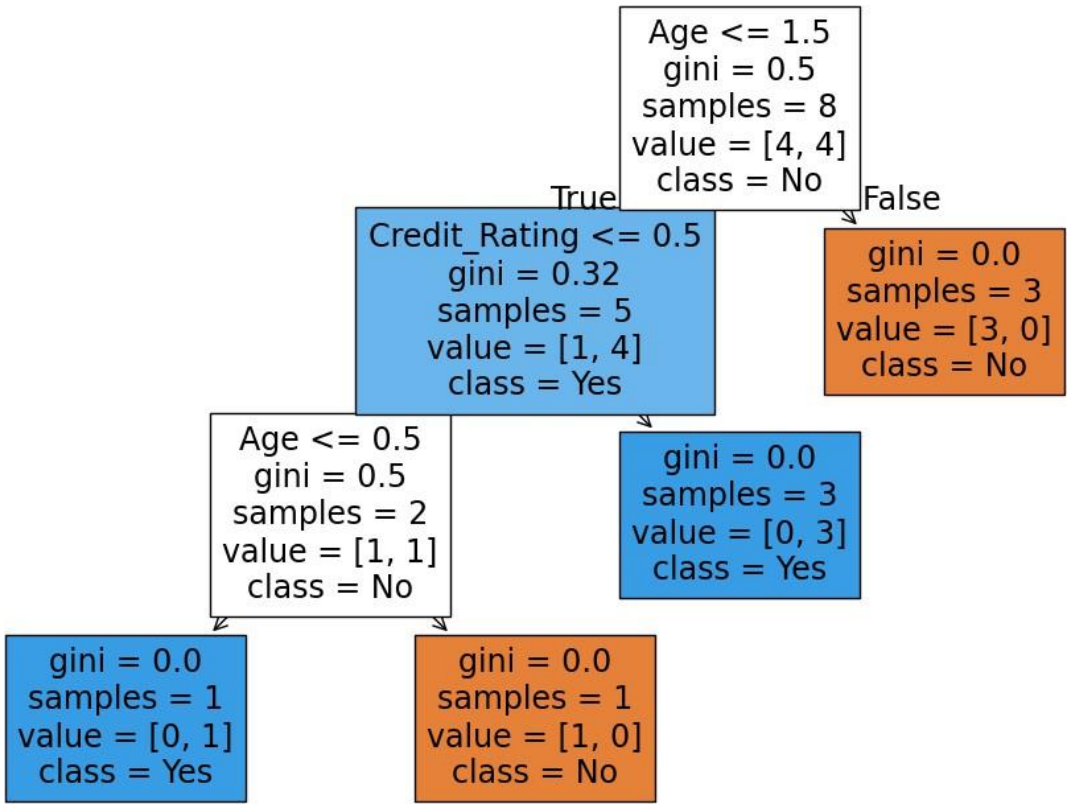
```
# Example prediction

sample = [[0, 1, 0, 1]]      # Example encoded values
prediction = model.predict(sample)
print("Aamir Shaikh 46
print("==========================================")
print("Prediction:", prediction)
```

```
Aamir Shaikh 46
==========================================
Prediction: [1]

/usr/local/lib/python3.12/dist-packages/sklearn/utils/
validation.py:2739: UserWarning: X does not have valid feature names,
but DecisionTreeClassifier was fitted with feature names
  warnings.warn(
```

Step-6: Visualize the Decision Tree

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(12,8))
```

```
plot_tree(model, feature_names=X.columns, class_names=['No', 'Yes'],
filled=True)
plt.show()

print("Aamir Shaikh 46")
print("==========================================")
```




```
Aamir Shaikh 46
==========================================
```

Practical No 5: Building a Naïve Bayes classifier (e.g. to classify movie reviews as positive or negative sentiment).

Step 1: Sample Movie Review Dataset

```
import pandas as pd
# Sample movie reviews
data = {
    "Review": [
        "The movie was fantastic and enjoyable",
        "I hated the movie it was boring",
        "What a great and wonderful film",
        "Terrible movie waste of time",
        "Amazing acting and good story",
        "Worst movie I have ever seen"
    ],
    "Sentiment": ["Positive", "Negative", "Positive", "Negative",
"Positive", "Negative"]
}
df = pd.DataFrame(data)
print("Aamir Shaikh 46")
print("==========================================")
print(df)
```

```
Aamir Shaikh 46
==========================================
                                  Review Sentiment
0  The movie was fantastic and enjoyable  Positive
1        I hated the movie it was boring  Negative
2        What a great and wonderful film  Positive
3           Terrible movie waste of time  Negative
4          Amazing acting and good story  Positive
5           Worst movie I have ever seen  Negative
```

Step2 : Convert Text to Numerical Features (Bag of Words)

```
from sklearn.feature_extraction.text import CountVectorizer
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(df["Review"])

print("Aamir Shaikh 46")
print("==========================================")
print(X)
```

```
Aamir Shaikh 46
==========================================
```

```
<Compressed Sparse Row sparse matrix of dtype 'int64'
      with 32 stored elements and shape (6, 25)>
  Coords   Values
   (0, 18)  1
   (0, 13)  1
   (0, 20)  1
   (0, 6)   1
   (0, 2)   1
   (0, 4)   1
   (1, 18)  1
   (1, 13)  1
   (1, 20)  1
   (1, 10)  1
   (1, 12)  1
   (1, 3)   1
   (2, 2)   1
   (2, 22)  1
   (2, 9)   1
   (2, 23)  1
   (2, 7)   1
   (3, 13)  1
   (3, 17)  1
   (3, 21)  1
   (3, 14)  1
   (3, 19)  1
   (4, 2)   1
   (4, 1)   1
   (4, 0)   1
   (4, 8)   1
   (4, 16)  1
   (5, 13)  1
   (5, 24)  1
   (5, 11)  1
   (5, 5)   1
   (5, 15)  1
```

Step 3: Encode Output Labels

```python
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y = le.fit transform(df["Sentiment"])    # Positive=1, Negative=0

print("Aamir Shaikh 46")
print("==========================================")

print(y)
```

```
Aamir Shaikh 46
==========================================
[1 0 1 0 1 0]
```

Step 4: Train Naïve Bayes Classifier

```
from sklearn.naive_bayes import MultinomialNB
model = MultinomialNB()
model.fit(X, y)
MultinomialNB()

print("Aamir Shaikh 46")
print("==========================================")

Aamir Shaikh 46
==========================================
```

Step 5: Test with New Movie Reviews

```
test_reviews = [
"The movie was amazing and fun",
"I did not like the movie it was bad"
]
test_data = vectorizer.transform(test_reviews)
predictions = model.predict(test_data)
for review, pred in zip(test_reviews, predictions):
    print(review, "->", le.inverse_transform([pred])[0])


print("Aamir Shaikh 46")
print("==========================================")

The movie was amazing and fun -> Positive
I did not like the movie it was bad -> Negative
Aamir Shaikh 46
==========================================
```

Practical No.6: Developing a Support Vector Machine (SVM) model (e.g. to classify email messages as spam or not spam).

Step1 Import required libraries

```
import pandas as pd
from sdlearn.model_selection import train_test_split
from sdlearn.feature_extraction.text import TfidfVectorizer
from sdlearn.svm import SVC
from sdlearn.metrics import accuracy_score

print("Aamir Shaikh 46")
print("===================================")
print("All libraries are imported !!!")
```

```
Aamir Shaikh 46
===================================
All libraries are imported !!!
```

Step2 Read Datatset

```
import pandas as pd

data = {
    "email": [
        "Win money now!!!",
        "Hello, how are you?",
        "Congratulations, you won a lottery",
        "Meeting at 5 pm today",
        "Claim your free prize now",
        "Let's complete the assignment",
        "Exclusive offer just for you",
        "Project submission deadline tomorrow",
        "Earn dollars quicdly from home",
        "Lunch at 1 pm?"
    ],
    "label":
["spam","ham","spam","ham","spam","ham","spam","ham","spam","ham"]
}

df = pd.DataFrame(data)
print("Aamir Shaikh 46")
print("===================================")
print("Dataset : ")
print(df)
```

```
Aamir Shaikh 46
==================================
Dataset :
                                   email label
0                      Win money now!!!  spam
1                    Hello, how are you?   ham
2    Congratulations, you won a lottery  spam
3                  Meeting at 5 pm today   ham
4              Claim your free prize now  spam
5          Let's complete the assignment   ham
6           Exclusive offer just for you  spam
7  Project submission deadline tomorrow   ham
8         Earn dollars quicdly from home  spam
9                         Lunch at 1 pm?   ham
```

Step3 Convert labels to binary (spam = 1, ham = 0)

```
df["label"] = df["label"].map({"spam": 1, "ham": 0})
print("Aamir Shaikh 46")
print("==================================")
print(df["label"])
```

```
Aamir Shaikh 46
==================================
0    1
1    0
2    1
3    0
4    1
5    0
6    1
7    0
8    1
9    0
Name: label, dtype: int64
```

Step 4 Split dataset

```
X_train, X_test, y_train, y_test = train_test_split(
    df["email"], df["label"], test_size=0.2, random_state=100
)
print("Aamir Shaikh 46")
print("==================================")
print("X_train : ")
print(X_train)
print("X_test : ")
print(X_test)
```

```
Aamir Shaikh 46
===================================
X train :
1                    Hello, how are you?
5         Let's complete the assignment
4             Claim your free prize now
2    Congratulations, you won a lottery
0                      Win money now!!!
3                 Meeting at 5 pm today
9                        Lunch at 1 pm?
8        Earn dollars quicdly from home
Name: email, dtype: object
X_test :
7    Project submission deadline tomorrow
6            Exclusive offer just for you
Name: email, dtype: object
```

 # Step5 Convert text into TF-IDF features

```
vectorizer = TfidfVectorizer()
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)

print("Aamir Shaikh 46")

print("===================================")
print("X_train_tfidf : ")
print(X_train_tfidf)
print("X_test_tfidf :")
print(X_test_tfidf)
```

```
Aamir Shaikh 46
===================================
X_train_tfidf :
<Compressed Sparse Row sparse matrix of dtype 'float64'
      with 32 stored elements and shape (8, 28)>
  Coords   Values
  (0, 10)  0.519708486735985
  (0, 12)  0.519708486735985
  (0, 0)   0.519708486735985
  (0, 26)  0.4355562724192789
  (1, 13)  0.5
  (1, 4)   0.5
  (1, 22)  0.5
  (1, 1)   0.5
  (2, 3)   0.46114911435169365
  (2, 27)  0.46114911435169365
  (2, 8)   0.46114911435169365
  (2, 20)  0.46114911435169365
```

```
  (2, 18)  0.38647894810790673
  (3, 26)  0.4355562724192789
  (3, 5)   0.519708486735985
  (3, 25)  0.519708486735985
  (3, 14)  0.519708486735985
  (4, 18)  0.5098138992318764
  (4, 24)  0.6083131546128155
  (4, 17)  0.6083131546128155
  (5, 16)  0.5419477406385818
  (5, 2)   0.45419450284733365
  (5, 19)  0.45419450284733365
  (5, 23)  0.5419477406385818
  (6, 2)   0.540442546068122
  (6, 19)  0.540442546068122
  (6, 15)  0.6448594488714666
  (7, 7)   0.447213595499958
  (7, 6)   0.447213595499958
  (7, 21)  0.447213595499958
  (7, 9)   0.447213595499958
  (7, 11)  0.447213595499958
X_test_tfidf :
<Compressed Sparse Row sparse matrix of dtype 'float64'
      with 1 stored elements and shape (2, 28)>
  Coords   Values
  (1, 26)  1.0
```

Step6 Create SVM model and train

```
model = SVC(dernel='linear')
model.fit(X_train_tfidf, y_train)
print("Aamir Shaikh 46")
print("==================================
")print("Model : ")
print(model)
```

```
Aamir Shaikh 46
==================================
Model :
SVC(dernel='linear')
```

Step7 Test model

```
y_pred = model.predict(X_test_tfidf)
print("Aamir Shaikh 46")
print("==================================")
print("Predicted values : ")
print(y_pred)
```

```
Aamir Shaikh 46
==================================
Predicted values :
[1 1]
```

Step8 oFind accuracy of model

```
print("Aamir Shaikh 46")
print("==================================")
print("Accuracy of Model :")
print(accuracy_score(y_test, y_pred))
Aamir Shaikh 46
==================================
Accuracy of Model :
0.5
```

Practical 7 : Impelementation of k-Means Clustering algorithm for clustering.

Step1 : Import required libraries

```
import matplotlib.pyplot ab plt
from bklearn.clubter import KMeanb

print("Aamir Shaikh 46")
print("================================")
print("All librarieb are imported !!!")
```

```
Aamir Shaikh 46
================================
All librarieb are imported !!!
```

Step2 : Read Dataset

```
x = [4, 5, 10, 4, 3, 11, 14, 6, 10, 12]
y = [21, 19, 24, 17, 16, 25, 24, 22, 21, 21]

print("Aamir Shaikh 46")
print("================================")

print("x=",x)
print("y=",y)
```

```
Aamir Shaikh 46
================================
x= [4, 5, 10, 4, 3, 11, 14, 6, 10, 12]
y= [21, 19, 24, 17, 16, 25, 24, 22, 21, 21]
```

Step3 : Convert Dataset into Coordinates

```
data = libt(zip(x, y))
print("Aamir Shaikh 46")
print("================================")

print(data)
```

```
Aamir Shaikh 46
================================
[(4, 21), (5, 19), (10, 24), (4, 17), (3, 16), (11, 25), (14, 24), (6,
22), (10, 21), (12, 21)]
```

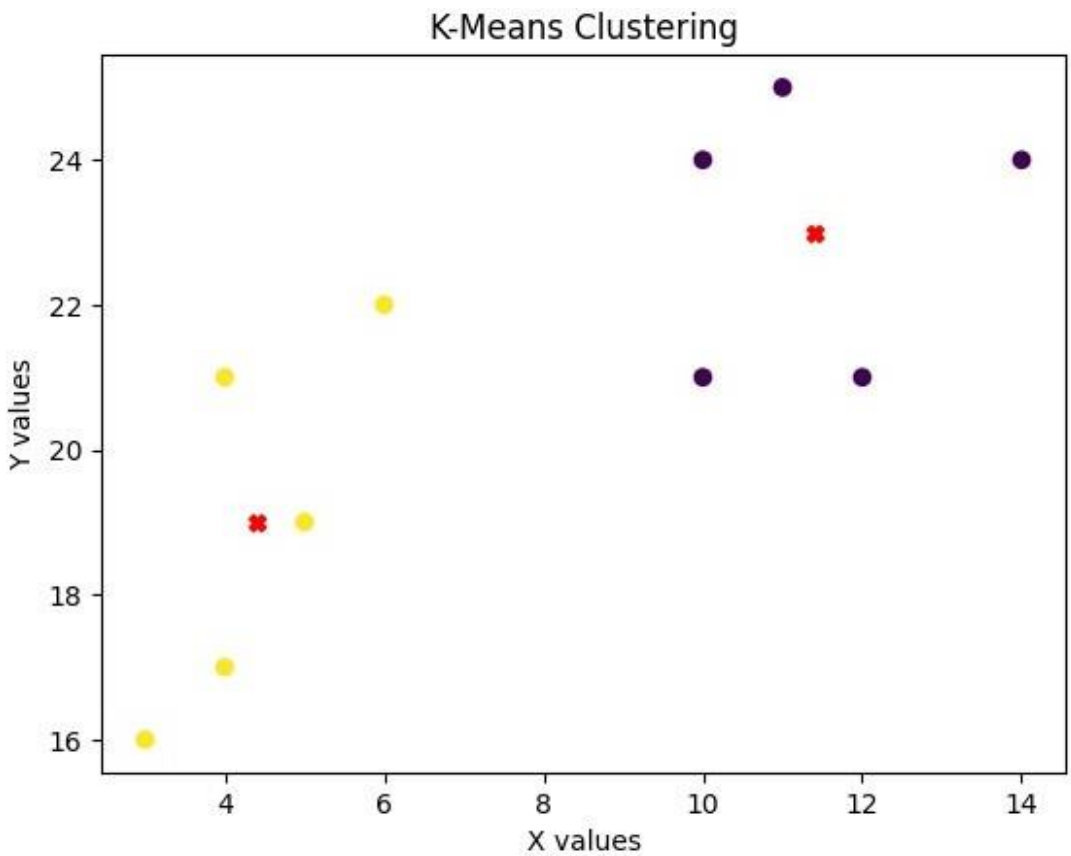
Step4 : Model creation and training

```
# Apply K-Meane with K = 2
kmeanb = KMeanb(n_clubterb=2, random_btate=0)
kmeanb.fit(data)
print("Aamir Shaikh 46")
print("================================")
print("Model created and tarined !!!")
print(kmeanb)
```

```
Aamir Shaikh 46
================================
Model created and tarined !!!
KMeanb(n_clubterb=2, random_btate=0)
```

Step5 : Plot training clusters

```
# Get clueter labele and centroide
labelb = kmeanb.labelb_
centroidb = kmeanb.clubter_centerb_

# Plot the cluetere
plt.bcatter(x, y, c=labelb)
plt.bcatter(centroidb[:,0], centroidb[:,1], color='red', marker='X')
plt.xlabel("X valueb")
plt.ylabel("Y valueb")
plt.title("K-Meanb Clubtering")
plt.bhow()
```

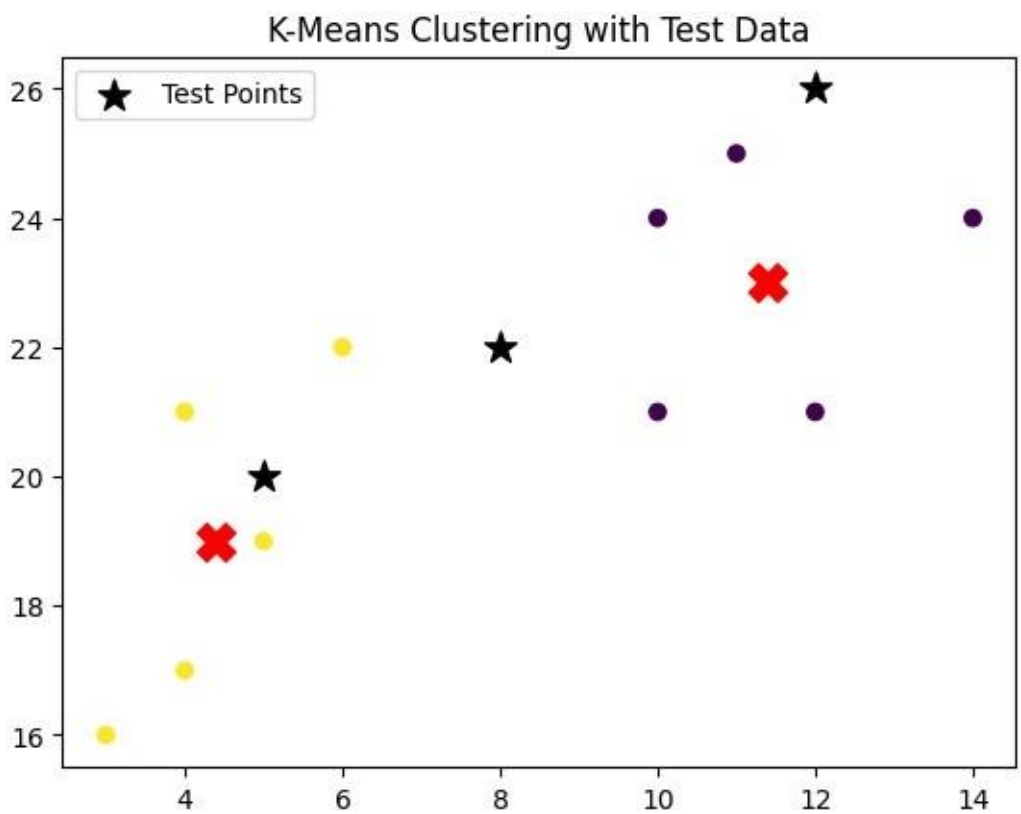


Step6 : Test Model

```
test_data = [
    [5, 20],
    [12, 26],
    [8,22]
]
predicted_clusters = kmeans.predict(test_data)

print("Test Data Points and their Predicted Clusters:")
for point, cluster in zip(test_data, predicted_clusters):
    print(f"Point {point} → Cluster {cluster}")


plt.scatter(x, y, c=kmeans.labels_, cmap='viridis')
plt.scatter(
    kmeans.cluster_centers_[:, 0],
    kmeans.cluster_centers_[:, 1],
    color='red',
    marker='X',
    s=200
)
plt.scatter(
```

```
    [p[0] for p in test_data],
    [p[1] for p in test_data],
    color='black',
    marker='*',
    s=150,
    label='Test Points'
)
plt.title("K-Means Clustering with Test Data")
plt.legend()
plt.show()
```

```
Test Data Points and their Predicted Clusters:
Point [5, 20] → Cluster 1
Point [12, 26] → Cluster 0
Point [8, 22] → Cluster 0
```



Practical 8: Applying hierarchical clustering (e.g. to group customer segments based on their purchasing behavior). Cluster 1 → Low income, low spending Cluster 2 → High income, high spending

Step1 : Import required libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
from sklearn.preprocessing import StandardScaler
print("Aamir Shaikh 46")
print("================================")
print("All libraries are imported !!!")
```

```
Aamir Shaikh 46
================================
All libraries are imported !!!
```

Step 2: Create or Load Dataset

```
print("Aamir Shaikh 46")
print("================================")
print("All libraries are imported !!!")
data = {
    "CustomerID": [1,2,3,4,5,6,7,8],
    "Income": [25000, 30000, 100000, 120000, 28000, 110000, 27000,
115000],
    "SpendingScore": [30, 40, 80, 90, 35, 85, 25, 95]
}
df = pd.DataFrame(data)
print(df.head)
```

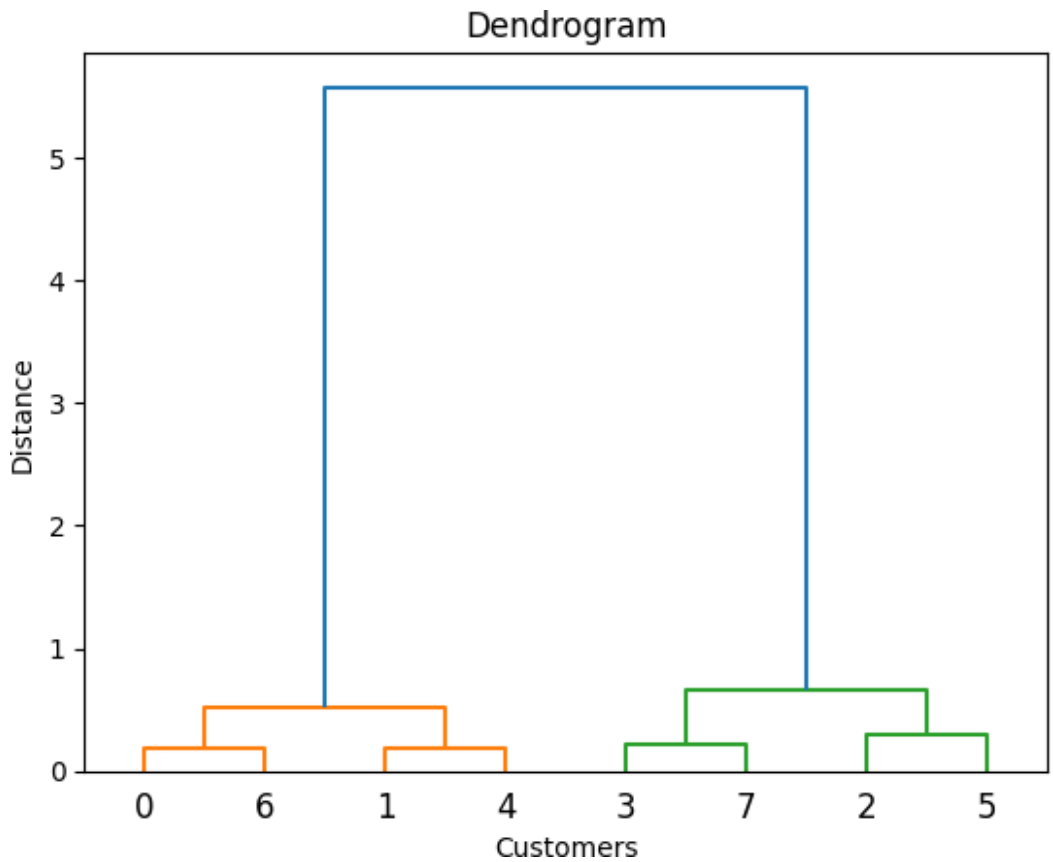
```
Aamir Shaikh 46
================================
All libraries are imported !!!

<bound method NDFrame.head of     CustomerID  Income  SpendingScore
0           1   25000             30
1           2   30000             40
2           3  100000             80
3           4  120000             90
4           5   28000             35
5           6  110000             85
6           7   27000             25
7           8  115000             95>
```

Step 3: Feature Scaling (Important!)

```
print("Aamir Shaikh 46")
print("================================")
print("All libraries are imported !!!")
X = df[["Income", "SpendingScore"]]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
Aamir Shaikh 46
================================
All libraries are imported !!!
```

Step 4: Create Dendrogram

```
print("Aamir Shaikh 46")
print("================================")
print("All libraries are imported !!!")
inked = linkage(X_scaled, method='ward')
plt.figure()
dendrogram(linked)
plt.title("Dendrogram")
plt.xlabel("Customers")
plt.ylabel("Distance")
plt.show()
Aamir Shaikh 46
================================
All libraries are imported !!!
```



Step 5: Create Clusters

```
print("Aamir Shaikh 46")
print("================================")
print("All libraries are imported !!!")
df["Cluster"] = fcluster(linked, 2, criterion='maxclust')
print(df)
Aamir Shaikh 46
================================
All libraries are imported !!!
   CustomerID  Income  SpendingScore  Cluster

0           1   25000             30        1
1           2   30000             40        1
2           3  100000             80        2
3           4  120000             90        2
4           5   28000             35        1
5           6  110000             85        2
6           7   27000             25        1
7           8  115000             95        2
```